Développement Efficace

Le type `Map`

Définition

Comme indiqué en cours, l'efficacité des tables de hachage dépend de nombreux paramètres : taille de la table, fonction de hachage, fonction de sondage ... Heureusement, de nombreuses implémentations sont disponibles. Notamment, TypeScript dispose du type `Map` qui permet d'associer des clefs et des valeurs.

Le type `Map` est un type **générique** c'est-à-dire qu'il marche pour tous les types qui sont hachables comme clef et tous les types comme valeurs. Un autre exemple de type générique que l'on a déjà vu sont les `array`. Quand on crée une nouvelle map, on doit donc indiquer le type des clefs et le types des données. Par exemple ci-dessous on utilise des nombres comme clef et des chaînes de caractères comme donnée.

```
let m = new Map<number, string>();
```

On peut utilise comme type pour les clefs tout type pour lequel on peut calculer une fonction de codage c'est-à-dire le calcul d'un nombre à partir de la valeur de la clef. On peut parfaitement utiliser des chaînes de caractères comme clef :

```
let m = new Map<string, number>();
```

Méthodes

Le type `Map` est une classe. On peut donc utiliser plusieurs méthodes sur les objets.

Méthodes	Descriptions
`map.set(key, value)`	ajoute une entrée dans la table
`map.get(key)`	retourne la valeur associée à la clef ou undefined si la clef n'existe pas
`map.has(key)`	retourne `true`si la clef est présente, `false` sinon
`map.delete(key)`	retire l'entrée associée à la clef. La méthode retourne `true` si une entrée associée à la clef existaient, `false` sinon
`map.size`	retourne la taille de la map
`map.clear()`	efface toutes les entrées de la map

Ci-dessous un exemple d'utilisation des `map` :

```
let m = new Map<string, number>();

m.set("George", 10);
m.set("Stevie", 18);
m.set("Eric", 15);
m.set("Ed", 5);

console.log('map.get("George") =', m.get("George"));
console.log('map.get("Eric") =', m.get("Eric"));
console.log('map.get("Jimi") =', m.get("Jimi"));
console.log('la clef "Eric" est-elle présente ?', m.has("Eric"));
console.log('la clef "Jimi" est-elle présente ?', m.has("Jimi"));
console.log("la taille est =", m.size);
console.log('On retire la clef "Ed" :', m.delete("Ed"));
console.log("maintenant la taille est =", m.size);
console.log('On retire la clef "Jimi" :', m.delete("Jimi"));
```

```
map.get("George") = 10
map.get("Eric") = 15
map.get("Jimi") = undefined
la clef "Eric" est-elle présente ? true
la clef "Jimi" est-elle présente ? false
la taille est = 4
On retire la clef "Ed" : true
maintenant la taille est = 3
On retire la clef "Jimi" : false
```

Itération sur les données

On peut itérer sur les clefs, les valeurs ou les entrées d'une map en utilisant une boucle `for...of`. Cela est illustré dans l'exemple ci-dessous sur la map `m` tel qu'elle est à la fin des instructions ci-dessus :

```
// Itération sur les clefs
console.log("Itération sur les clefs");
for (let key of m.keys()) console.log("Clef =", key);

// Itération sur les valeurs
console.log("\nItération sur les valeurs");
for (let value of m.values()) console.log("Valeur =", value);

//Itération sur les entrées
console.log("\nItération sur les entrées");
for (let entry of m.entries()) console.log(entry[0], entry[1]);
```

```
Itération sur les clefs
Clef = George
Clef = Stevie
Clef = Eric

Itération sur les valeurs
Valeur = 10
Valeur = 18
Valeur = 15

Itération sur les entrées
George 10
Stevie 18
Eric 15
```

Exemple d'utilisation d'une map comme compteur

Problèmatique : Etant donné une chaîne de caractères constituées des 26 lettres de l'alphabet en minuscule, on souhaite compter combien de fois chaque lettre présente dans la chaîne apparaît.

Solutions possibles :

1. Créer 26 variables initialisées à 0, une pour chaque lettre, parcourir la chaîne et incrémenter les variables selon les caractères rencontrés.
2. Créer un tableau de 26 entiers initialisée avec de 0, parcourir la chaîne est incrémenter l'élement correspondant (0 pour 'a', 1 pour 'b' ... cet indice peut être calculé automatiquement)
3. Créer un dictionnaire vide qui utilise les caractères comme des clefs, parcourir la liste, si le caractère courant ne sert pas déjà de clef, on crée une nouvelle paire clef-valeur (avec 1 pour valeur), sinon on incrémente la valeur associée à la clef de 1.

Remarque : la solution basée sur les dictionnaires marche pour toutes les chaînes de caractères quelque soit les caractères présents sans modification.

Remarque : un caractère est une chaîne de caractères de longueur 1.

```
function comptage(chaîne: string): Map<string, number> {
  let m = new Map<string, number>();

  for (let c of chaîne)
    if (m.has(c)) m.set(c, m.get(c)! + 1);
    else m.set(c, 1);

  return m;
}

let m = comptage("brontosaurus");
console.log(m);
```

```
Map(8) {
  'b' => 1,
  'r' => 2,
  'o' => 2,
  'n' => 1,
  't' => 1,
  's' => 2,
  'a' => 1,
  'u' => 2
}
```

Exercices

Exercice 1

Dans cet exercice, on suppose qu'une chaîne de caractères est constituée de *mots* séparées par des espaces. Par exemple, la chaîne `"Ceci est une chaine et cette chaîne est constituée de dix mots"` contient les mots `"Ceci"`, `"est"`, `"une"`, `"chaîne"`, `"et"`, `"cette"`, `"constituée"`, `"de"`, `"dix"`, `"mots"`.

Ecrivez une fonction qui prend une chaîne de caractères et qui retourne sous forme de map les mots qu'elle contient et combien de fois ils apparaîssent.

Exercice 2

Ecrivez une fonction qui prend en paramètre un tableau de nombres et qui retourne `true` si et seulement si elle contient au moins un élément qui apparaît plusieurs fois. Vous devez utiliser une map dans votre fonction (il existe des solutions sans utiliser de dictionnaire).

Exercice 3

Pour rappel, la liste de Fibonnacci est définie par :

$$u_n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ u_{n-1} + u_{n-2} & \text{sinon} \end{cases}$$

Son implémentation récursive est la suivante :

```
function fibonacci(n: number): number {
  if (n === 0) return 0;
  else if (n === 1) return 1;
  else return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Cette version devient vite inefficace quand n grandit. Le problème est que l'on calcule plusieurs fois la même valeur de la suite pour un n donné. Un manière de contourner ce problème est de stocker les valeurs de la suite déjà calculées dans une map dont les clefs sont les valeurs de n. Ainsi, quand on veut calculer u_n, on regarde si la valeur est présente dans la map. Si c'est le cas, on retourne la valeur associée à la clef n. Sinon, on fait l'appel récursif normal et on récupère le résultat puis on le stocke dans la map avant de le retourner. Ce principe s'appelle la **mémoïsation**.

Compléter la fonction ci-dessous, la map `mémoire` sert à stocker les valeurs connues, initialialement on connaît u_0 et u_1.

```
let mémoire = new Map<number, number>();
mémoire.set(0, 0);
mémoire.set(1, 1);

function fibonacci(n: number, mémoire: Map<number, number>): number {
  // Tester ici si on connaît déjà le résultat pour n, si oui le retourner
  // Sinon, faire l'appel récursif
  // Stocker le résultat dans mémoire
  // Retourner le résultat
}
```

Exercice 4

Ecrivez une version mémoïsée de la fonction d'Ackermann (cf le cours sur la récursivité) :

$$A(m,n) = \begin{cases} n+1 & \text{si } m = 0 \\ A(m-1,1) & \text{si } m > 0 \text{ et } n = 0 \\ A(m-1, A(m, n-1)) & \text{si } m > 0 \text{ et } n > 0 \end{cases}$$

Indication : il n'est pas possible d'utiliser une référence sur un objet comme clef. Ici, vous pouvez utiliser une chaîne de caractères. Cette dernière doit être unique pour chaque paire (m, n). Par exemple, on peut utiliser la chaîne `"m,n"`.

Exercice 5

On souhaite écrire une fonction qui prend un tableau de nombres et qui retourne le nombre qui apparaît le plus dans le tableau et le nombre fois que ce nombre apparaît. La solution doit utiliser une map.

Indication : pour définir une fonction qui retourne deux valeurs, on peut utiliser un **tuple**. Par exemple, la fonction ci-dessous retourne dans un tuple le quotient et le reste de la division entière de `a` par `b`.

```
function divisionEntière(a: number, b: number): [number, number] {
  return [Math.floor(a / b), a % b];
}

let résultat = divisionEntière(1872, 133);
```

```
console.log("quotient =", résultat[0]);
console.log("reste =", résultat[1]);
```

```
quotient = 14
reste = 10
```